

Extremely Detailed Explanation of Diff3Det

Diffusion-based 3D Object Detection with Random Boxes

Contents

1	Introduction and Central Motivation	4
2	3D Object Detection Background	4
2.1	Why LiDAR is useful	4
2.2	Point-based 3D detection	4
2.3	Voxel-based 3D detection	5
2.4	Voxelization versus voxel encoding	5
2.5	Sparse convolution	5
2.6	Why some methods use BEV	6
2.7	Information loss from voxelization	6
3	Image-based and Multimodal 3D Detection	6
4	Why Proposal Generation Matters	6
5	Diffusion Models	7
5.1	Forward process	7
5.2	Reverse process	7
5.3	Diffusion for object detection	7
6	Overall Diff3Det Pipeline	7
7	BEV Proposal Representation	8
8	Forward Diffusion of Bounding Boxes	8
8.1	Meaning of x_0	8
8.2	Meaning of ϵ	8
8.3	Meaning of the diffusion timestep	8
9	Empty Proposals and Resampling	9

9.1 Why a BEV box can be checked against LiDAR points	9
10 Correlation Constraint on Width and Length	9
11 Dynamic Time Step	10
12 BEV Features, Proposal Boxes, and RoI Extraction	11
13 Time Embedding and Detection Decoder	11
14 Bipartite Matching and Loss	11
15 Inference and Reverse Diffusion	12
16 Multiple Sampling Steps and NMS	13
17 Ablation Study: Proposal Components	13
17.1 Baseline	13
17.2 Corrupted proposals from ground truth	13
17.3 Resampling	14
17.4 Size correlation	14
17.5 Dynamic timestep	14
18 Sampling-Step Ablation	15
19 Hyperparameter Ablation: Signal Scale	15
20 Hyperparameter Ablation: Number of Proposals	16
21 Hyperparameter Ablation: Resampling Threshold	16
22 Why the Four Proposal Refinements Complement Each Other	17
23 Why Random Boxes Can Work	17
24 Diffusion is Not Replacing the Detector	17
25 Why BEV is Suitable for Proposal Generation	18
26 Complete Mathematical Pipeline	18
27 Integrated Conceptual Walkthrough	19
28 Most Important Conceptual Comparison	19

1 Introduction and Central Motivation

Diff3Det is a 3D object detection framework whose main novelty is the use of a diffusion process for generating object proposals. The central problem is not merely how to classify or regress an object once a good candidate exists; it is how to generate useful candidate boxes in the first place.

A conventional detector can be viewed conceptually as

LiDAR point cloud $\rightarrow$ feature extraction $\rightarrow$ proposal generation $\rightarrow$ classification and regression $\rightarrow$ final 3D boxes

Traditional 3D detectors often use anchor boxes. An anchor configuration may specify different sizes, aspect ratios, orientations, and spatial placements before learning begins. The designer therefore has to make decisions about what candidate objects should look like.

Diff3Det asks whether proposals can instead be generated by a learned generative process. Its answer is diffusion: noisy/random boxes are progressively transformed into boxes corresponding to actual objects.

The central conceptual change is

hand-designed proposal generation $\longrightarrow$ learned diffusion-based proposal generation

.

Diffusion does not replace the whole detector. The LiDAR encoder still extracts scene features, and the detection decoder still predicts classes and refined boxes. Diffusion is primarily used for proposal generation.

2 3D Object Detection Background

2.1 Why LiDAR is useful

A LiDAR sensor provides a point cloud containing measurements in three-dimensional space. A point can be represented as

$$p_i = (x_i, y_i, z_i),$$

and a complete point cloud as

$$P = \{p_i\}_{i=1}^N.$$

Depending on the sensor, points can also contain intensity or reflectance.

The main advantage of LiDAR is that geometry and depth are measured directly. A 3D detector wants object position, dimensions, orientation, and depth, and LiDAR provides direct geometric evidence for these quantities.

2.2 Point-based 3D detection

Point-based methods operate directly on the unstructured point cloud. Unlike an image, the points are not naturally arranged in a dense rectangular grid.

An image has a regular structure such as

pixel pixel pixel
pixel pixel pixel,
pixel pixel pixel

whereas LiDAR points may occur at arbitrary spatial locations.

Point-based methods preserve point-level geometry because they do not first quantize the scene into voxels. However, directly processing a very large number of points can be expensive in memory and computation, particularly for neighborhood construction and feature aggregation. Thus point-based methods preserve fine information but can be less efficient.

2.3 Voxel-based 3D detection

Voxel-based methods divide 3D space into small cuboidal cells called voxels. A voxel can be indexed by

$$V_{ijk},$$

where i, j, k correspond to the three spatial directions.

The transformation is

$$\boxed{\text{irregular point cloud} \rightarrow \text{regular voxel grid}}.$$

Once the point cloud has been voxelized, convolutional processing becomes much more natural. This is the basic idea behind VoxelNet:

$$\boxed{\text{Point cloud} \rightarrow \text{Voxel grid} \rightarrow \text{3D feature extraction}}.$$

2.4 Voxelization versus voxel encoding

These two terms should not be confused.

Voxelization assigns each LiDAR point to a spatial grid cell. It answers: which voxel contains this point?

A **voxel encoder** processes the points/features associated with occupied voxels and produces learned voxel features:

$$\text{points in voxel} \rightarrow \text{voxel feature}.$$

A larger **3D voxel backbone** can then process these features and aggregate spatial context. Different papers use “voxel encoder,” “voxel backbone,” and “3D encoder” at slightly different levels of abstraction.

For Diff3Det, the safest conceptual pipeline is

$$\boxed{\text{LiDAR} \rightarrow \text{voxelization} \rightarrow \text{3D voxel encoder/backbone} \rightarrow \text{BEV features}}.$$

2.5 Sparse convolution

A voxel grid can contain a huge number of possible cells while only a small fraction contain LiDAR points. For example,

$$200 \times 200 \times 20 = 800000$$

possible voxel locations can exist, most of which may be empty.

Dense 3D convolution wastes computation on these empty locations. Submanifold sparse convolution instead performs computation primarily at active locations, greatly improving efficiency.

2.6 Why some methods use BEV

Autonomous-driving objects are predominantly distributed across the road plane. They are not normally stacked repeatedly at exactly the same horizontal location. This motivates a top-down representation.

A LiDAR point has coordinates

$$(x, y, z),$$

while BEV primarily indexes the horizontal plane

$$(x, y).$$

BEV should be understood as a learned top-down representation derived from the 3D LiDAR scene, not as the raw LiDAR itself. Height information can still be encoded inside the feature channels even though the spatial indexing is two-dimensional.

The benefit is that subsequent spatial processing can be performed efficiently in 2D.

2.7 Information loss from voxelization

Voxelization introduces quantization. Suppose

$$p_1 = (1.01, 2.01, 1.20), \quad p_2 = (1.09, 2.08, 1.25).$$

If both points fall into the same voxel, their exact continuous positions are no longer preserved in the same way as in the raw point cloud.

Thus

$\text{continuous point cloud} \rightarrow \text{discrete grid}$

can lose geometric detail.

This motivates hybrid point-plus-voxel methods: voxel representations provide efficient feature extraction, while point representations preserve fine geometric information.

3 Image-based and Multimodal 3D Detection

Camera-only 3D detection is attractive because cameras are cheaper than LiDAR. However, an image directly provides 2D appearance and coordinates but does not directly provide depth. A small object far away can have an image size similar to a large object nearby, producing depth ambiguity.

Multimodal approaches combine

$\text{LiDAR} + \text{camera}$.

LiDAR provides geometry and depth, while the camera provides rich semantic and appearance information. Their combination can yield strong performance.

4 Why Proposal Generation Matters

Traditional detectors often require predefined anchors. An anchor can specify a candidate box size, aspect ratio, orientation, and location strategy. The designer therefore chooses what types of candidate boxes the detector will examine.

Diff3Det seeks a learned alternative:

diffusion-based proposal generation

.

5 Diffusion Models

5.1 Forward process

Diffusion starts from a meaningful sample x_0 and progressively adds Gaussian noise:

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_T.$$

At a large timestep,

$$x_T \approx \mathcal{N}(0, I).$$

Thus,

structured sample $\rightarrow$ noise

.

5.2 Reverse process

A neural network learns to reverse the corruption:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

Thus,

noise $\rightarrow$ structured sample

.

The key idea is to learn how to remove corruption.

5.3 Diffusion for object detection

DiffusionDet showed that diffusion can generate bounding boxes. Instead of making

$$x = \text{image},$$

the diffusion variable can be

$$x = \text{bounding box}.$$

Hence

noise $\rightarrow$ object proposal

.

Diff3Det extends this idea to 3D object detection using BEV box parameters.

6 Overall Diff3Det Pipeline

The complete pipeline is

LiDAR point cloud

 $\rightarrow$ voxelization $\rightarrow$ 3D voxel encoder/backbone $\rightarrow$ BEV feature map

followed by

diffusion proposal generator $\rightarrow$ BEV proposals $\rightarrow$ BEV RoI features $\rightarrow$ dynamic detection decoder $\rightarrow$ cl

The three major conceptual components are:

1. **Encoder:** extracts features from the LiDAR point cloud.
2. **Diffusion-guided proposal generator:** generates candidate boxes.
3. **Decoder:** uses proposal features and diffusion-time information to predict classes and refined boxes.

7 BEV Proposal Representation

Diff3Det performs diffusion on

$$x = (c_x, c_y, d_x, d_y, \theta).$$

Here c_x, c_y are the BEV center coordinates, d_x, d_y are the horizontal dimensions, and θ is orientation.

Thus the diffusion variable is a five-dimensional vector. It is not an image and does not contain pixels. It directly represents box geometry.

Ground-truth boxes are repeated to obtain the desired proposal count and normalized, for example, to $[0, 1]$.

8 Forward Diffusion of Bounding Boxes

The paper uses

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \tag{4}$$

where

$$\epsilon \sim \mathcal{N}(0, I_5).$$

8.1 Meaning of x_0

x_0 is the clean ground-truth box. For example,

$$x_0 = [0.52, 0.37, 0.18, 0.08, 0.40]^T.$$

8.2 Meaning of ϵ

Because the box has five parameters,

$$\epsilon = [\epsilon_1, \epsilon_2, \epsilon_3, \epsilon_4, \epsilon_5]^T, \quad \epsilon_i \sim \mathcal{N}(0, 1).$$

8.3 Meaning of the diffusion timestep

The coefficient $\bar{\alpha}_t$ controls the relative amount of retained signal and noise. At lower noise levels the corrupted box remains close to x_0 ; at higher noise levels it becomes increasingly random.

The qualitative relation is

$$t \uparrow \Rightarrow \text{more corruption}.$$

During training,

$$t = \text{randint}(1, T_{\max}),$$

so the decoder sees many different noise levels.

9 Empty Proposals and Resampling

A random proposal can cover a region with very few or no LiDAR points. Its corresponding RoI feature can then contain little useful geometric evidence.

Let

$m = \text{number of LiDAR points associated with the proposal footprint.}$

If

$$m < \eta,$$

the proposal is discarded and resampled. Sampling continues until

$$\boxed{m \geq \eta}.$$

The reported best value is

$$\boxed{\eta = 5}.$$

9.1 Why a BEV box can be checked against LiDAR points

The raw LiDAR point is

$$(x_i, y_i, z_i).$$

The BEV proposal describes a region in the physical horizontal coordinate system. Therefore the proposal footprint can be tested against the horizontal coordinates

$$(x_i, y_i).$$

This is not saying that BEV and LiDAR are the same representation. Instead:

BEV is derived from LiDAR, and both share the physical (x, y) coordinate system.

There are two distinct operations:

1. **Point-count filtering:** count original LiDAR points whose horizontal locations fall inside the proposal footprint.
2. **RoI extraction:** use the accepted proposal box to extract its corresponding region from the learned BEV feature map.

Thus “at least five LiDAR points” does not mean five BEV feature pixels.

10 Correlation Constraint on Width and Length

A naive random generator might sample width and length independently. However, realistic object dimensions exhibit statistical relationships. The paper therefore uses correlated random variables.

The proposed construction is

$$\boxed{W = \rho L + \sqrt{1 - \rho^2} X} \tag{5}$$

where

$$L, X \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$$

and

$$\rho = 0.8.$$

If $\rho = 0$,

$$W = X,$$

so W and L are independent.

If $\rho = 1$,

$$W = L,$$

so they are perfectly correlated.

For $\rho = 0.8$,

$$W = 0.8L + 0.6X.$$

Thus most of the variation in W is linked to L , while some independent variation remains.

The generated variables are then scaled to appropriate ranges. The paper reports

$$w = 8, \quad l = 5.$$

The important principle is

$$\boxed{\text{random} \neq \text{unconstrained random}}.$$

The method injects a useful geometric prior.

11 Dynamic Time Step

A highly corrupted box is difficult to recover, especially early in training when the network is weak. Diff3Det therefore increases the maximum available noise level over training.

This resembles curriculum learning:

$$\boxed{\text{low-noise denoising} \rightarrow \text{high-noise denoising}}.$$

The reported sine schedule is

$$T_{\max} = \begin{cases} T \left\lfloor \sin \left(\cos^{-1} \left(\frac{\omega}{T} \right) \frac{x}{\sigma n} + \sin^{-1} \left(\frac{\omega}{T} \right) \right) \right\rfloor, & x < \sigma n, \\ T, & x \geq \sigma n. \end{cases} \quad (6)$$

The reported hyperparameters are

$$\omega = 5, \quad \sigma = 0.5.$$

Here x is the current epoch, n is the total number of epochs, and T is the final maximum timestep.

Conceptually, early training uses a small range of timesteps, and the range gradually expands until the full diffusion range is available.

12 BEV Features, Proposal Boxes, and RoI Extraction

The encoder produces a BEV feature map

$$F_{\text{BEV}}(x, y) \in \mathbb{R}^C.$$

Suppose diffusion produces a proposal

$$B = (c_x, c_y, d_x, d_y, \theta).$$

The proposal is used to select its corresponding region in the BEV feature map:

$$\boxed{(F_{\text{BEV}}, B) \rightarrow F_{\text{RoI}}}.$$

This is what the paper means by “BEV proposal features.”

The key distinction is:

$$\boxed{\text{raw LiDAR points} \neq \text{BEV feature map}}$$

but the BEV feature map was learned from the LiDAR data.

13 Time Embedding and Detection Decoder

The decoder receives the proposal-specific feature and a diffusion-time embedding:

$$\boxed{F_{\text{RoI}} + \text{Embed}(t) \rightarrow \text{dynamic decoder}.}$$

The time embedding tells the decoder how noisy the current proposal is. A proposal at low t is relatively close to a clean box, while a proposal at high t is much more corrupted.

The decoder predicts both:

- **classification:** what object category is present?
- **regression:** how should the box be refined?

Thus

$$(F_{\text{RoI}}, t) \rightarrow (\hat{c}, \hat{x}).$$

14 Bipartite Matching and Loss

Let the ground-truth set be

$$y = \{y_i\}_{i=1}^M$$

and predictions be

$$\hat{y} = \{\hat{y}_j\}_{j=1}^N.$$

Since M and N need not be equal, predictions must be matched to ground truth.

The matching cost is

$$\boxed{C_{\text{match}} = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{IoU}} L_{\text{BEV IoU}}} \quad (7)$$

It therefore considers class correctness, geometric regression error, and overlap.

The classification loss is focal loss.

The regression loss is an L_1 loss:

$$L_{\text{reg}} = \sum_k |\hat{x}_k - x_k|.$$

For

$$x = [c_x, c_y, d_x, d_y, \theta]^T,$$

this becomes

$$L_1 = |\hat{c}_x - c_x| + |\hat{c}_y - c_y| + |\hat{d}_x - d_x| + |\hat{d}_y - d_y| + |\hat{\theta} - \theta|.$$

IoU is

$$\text{IoU} = \frac{\text{intersection}}{\text{union}}.$$

For BEV boxes this describes top-down overlap. The final loss uses a rotated 3D DIoU-based loss:

$$L_{\text{DIoU}}.$$

The final training objective is

$$\boxed{L = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{IoU}} L_{\text{DIoU}}} \quad (9)$$

with

$$\lambda_{\text{cls}} = 2, \quad \lambda_{\text{reg}} = 5, \quad \lambda_{\text{IoU}} = 2.$$

Therefore,

$$\boxed{L = 2L_{\text{cls}} + 5L_{\text{reg}} + 2L_{\text{DIoU}}}.$$

15 Inference and Reverse Diffusion

At inference there is no ground truth. The process therefore starts from Gaussian random proposals:

$$x_T \sim \mathcal{N}(0, I).$$

The proposals are progressively denoised:

$$x_T \rightarrow x_{T-s} \rightarrow x_{T-2s} \rightarrow \cdots \rightarrow x_0.$$

The conceptual operation at each step is

$$\boxed{\text{current noisy proposal} \rightarrow \text{estimate corruption} \rightarrow \text{remove corruption} \rightarrow \text{cleaner proposal}.}$$

The paper’s sampling equation is

$$\boxed{x_{t-s} = \sqrt{\alpha_{t-s}} \left(\frac{x_t - \sqrt{1 - \alpha_t} \epsilon_{\theta}^{(t)}(x_t)}{\sqrt{\alpha_t}} \right) + \sqrt{1 - \alpha_{t-s} - \sigma_t^2} \epsilon_{\theta}^{(t)}(x_t) + \sigma_t \epsilon_t} \quad (10)$$

with

$$\sigma_t = \sqrt{\frac{1 - \alpha_t / \alpha_{t-s}}{(1 - \alpha_{t-s}) / (1 - \alpha_t)}}. \quad (11)$$

Here $\epsilon_\theta^{(t)}(x_t)$ represents the network-predicted noise/offset for the current proposal and timestep, while ϵ_t is Gaussian noise.

16 Multiple Sampling Steps and NMS

One sampling step makes a large correction:

$$x_{1000} \rightarrow x_0.$$

Four steps make smaller corrections:

$$x_{1000} \rightarrow x_{750} \rightarrow x_{500} \rightarrow x_{250} \rightarrow x_0.$$

More steps can improve refinement initially, but they increase computation and do not guarantee monotonic performance improvement.

Repeated proposals can converge to the same object. Therefore NMS removes redundant predictions by keeping high-confidence boxes and suppressing highly overlapping duplicates.

17 Ablation Study: Proposal Components

Table 1: Proposal-generation ablation.

Component	Easy	Moderate	Hard	mAP
Baseline	82.62	74.21	73.56	76.80
+ Corrupted proposals from GT	84.32	76.17	74.41	78.30
+ Resample	85.31	76.31	74.48	78.70
+ Size correlation	86.14	76.80	75.29	79.41
+ Dynamic time step	87.84	77.90	76.07	80.61

17.1 Baseline

Random Gaussian proposals give

$$\boxed{76.80 \text{ mAP}}.$$

This is the reference system.

17.2 Corrupted proposals from ground truth

Training with noisy versions of ground-truth boxes improves

$$76.80 \rightarrow 78.30,$$

a gain of

$$\boxed{+1.50 \text{ percentage points}}.$$

The network now learns a meaningful mapping:

$$\text{corrupted real object} \rightarrow \text{real object}.$$

17.3 Resampling

Requiring enough LiDAR points gives

$$78.30 \rightarrow 78.70,$$

a gain of

$$\boxed{+0.40}.$$

The improvement supports the idea that proposals with little geometric evidence produce poor RoI features.

17.4 Size correlation

Correlated width and length improve

$$78.70 \rightarrow 79.41,$$

a gain of

$$\boxed{+0.71}.$$

This demonstrates that proposal quality matters: random proposals are more useful when their geometry resembles plausible objects.

17.5 Dynamic timestep

Dynamic timestep improves

$$79.41 \rightarrow 80.61,$$

a gain of

$$\boxed{+1.20}.$$

The model benefits from learning easy denoising tasks before harder denoising tasks.

Overall,

$$76.80 \rightarrow 80.61,$$

which is

$$\boxed{+3.81 \text{ percentage points}}.$$

18 Sampling-Step Ablation

Table 2: Sampling-step ablation.

Steps	Easy	Moderate	Hard	mAP
1	89.29	79.91	75.48	81.56
2	89.56	80.26	76.35	82.06
4	89.45	80.86	77.41	82.57
6	89.43	80.28	76.76	82.16
8	88.76	80.24	77.24	82.08

Four steps give the best reported mAP:

82.57.

The progression from one to four steps suggests that iterative refinement is useful. Beyond four, extra computation does not guarantee further improvement.

The authors also distinguish AP40 from AP11 evaluation. Their discussion emphasizes that AP40 better reflects the iterative-sampling benefit in this experiment. Consequently, a small decrease under another evaluation protocol should not automatically be interpreted as evidence that iterative sampling is ineffective.

19 Hyperparameter Ablation: Signal Scale

Table 3: Signal-scale ablation.

Scale	Easy	Moderate	Hard
mAP			
0.1	15.36	14.66	14.35
	14.79		
1.0	83.87	74.50	73.44
	77.27		
2.0	87.84	77.90	76.07
	80.61		

The best tested value is

scale = 2.0.

At scale 0.1, mAP collapses to 14.79, showing that the diffusion signal/noise relationship is badly conditioned for this setting. At 1.0, performance becomes much better, and 2.0 gives the strongest result among the tested values.

The important concept is balance:

useful retained structure + sufficient corruption.

20 Hyperparameter Ablation: Number of Proposals

Table 4: Proposal-number ablation.

N	Easy	Moderate	Hard
mAP			
100	86.59	77.09	75.75
79.81			
300	87.84	77.90	76.07
80.61			
500	86.25	76.91	75.53
79.63			

The best value is

$$\boxed{N = 300}.$$

With only 100 proposals, fewer candidate regions are available, potentially reducing object coverage and recall.

With 500 proposals, computation and redundancy increase. More proposals also create more candidate predictions and do not automatically produce better coverage. Therefore there is an optimum, and 300 performs best in the reported experiment.

21 Hyperparameter Ablation: Resampling Threshold

Table 5: Resampling-threshold ablation.

η	Easy	Moderate	Hard
mAP			
1	86.44	77.22	75.78
79.81			
5	87.84	77.90	76.07
80.61			
10	86.69	77.38	76.05
80.04			

The best value is

$$\boxed{\eta = 5}.$$

At $\eta = 1$, a proposal can be accepted with only one LiDAR point. Such a point gives very weak information about object shape, size, orientation, and extent.

At $\eta = 10$, the criterion can become too restrictive. Faraway objects, sparse LiDAR regions, partially occluded objects, or proposals covering only part of an object may naturally contain fewer than ten points. Thus $\eta = 10$ can reject useful candidates.

Five points provide the best balance in the reported experiments.

22 Why the Four Proposal Refinements Complement Each Other

The proposal quality can be viewed conceptually as depending on several factors:

$$Q = f(\text{location, LiDAR evidence, shape, noise level}).$$

Each proposed component addresses a different weakness:

Component	Purpose
Corrupted GT proposals	Makes training proposals directly related to real objects.
Resampling	Removes candidates with insufficient LiDAR evidence.
Size correlation	Makes width-length combinations more realistic.
Dynamic timestep	Gradually increases denoising difficulty during training.

Thus the components are not arbitrary tricks. They address different failure modes of naive random proposals.

23 Why Random Boxes Can Work

The word “random” refers primarily to the initialization, not the final result.

The initial proposal can be poor, but the model has access to scene-conditioned LiDAR features. The decoder learns how to use those features to move the proposal toward a meaningful object.

A useful conceptual relationship is

$$\text{random initial box} + \text{LiDAR-derived feature} + \text{diffusion timestep} \rightarrow \text{refined object box}.$$

Therefore the diffusion process does not need the initial random box to already be a good detection. It needs the learned reverse process to transform the candidate into one.

24 Diffusion is Not Replacing the Detector

The architecture should not be interpreted as “diffusion does all detection.” Instead,

$$\text{3D encoder} + \text{diffusion proposal generator} + \text{detection decoder}.$$

The encoder extracts scene features.

The diffusion module generates/refines candidate boxes.

The decoder predicts class and box corrections.

The novelty is mainly the learned proposal-generation mechanism.

25 Why BEV is Suitable for Proposal Generation

The diffusion variable

$$(c_x, c_y, d_x, d_y, \theta)$$

is naturally a horizontal-plane representation. Object centers and horizontal dimensions are conveniently represented in BEV.

The 3D encoder has already extracted information from the full 3D LiDAR structure. Therefore BEV provides an efficient spatial representation for proposal generation while retaining useful 3D information in its learned features.

The conceptual flow is

$3\text{D LiDAR geometry} \rightarrow \text{BEV learned features} \rightarrow 2\text{D proposal geometry} \rightarrow 3\text{D-aware detection.}$

26 Complete Mathematical Pipeline

The complete system can be summarized mathematically as follows.

Input:

$$P = \{(x_i, y_i, z_i)\}_{i=1}^N.$$

Voxelization:

$$P \rightarrow V.$$

3D encoder:

$$V \rightarrow F_{\text{BEV}}.$$

Training proposal generation:

$$x_0 \rightarrow x_t$$

with

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon.$$

Proposal-quality filtering:

$$m \geq \eta, \quad \eta = 5.$$

RoI extraction:

$$(F_{\text{BEV}}, x_t) \rightarrow F_{\text{RoI}}.$$

Decoder:

$$(F_{\text{RoI}}, t) \rightarrow (\hat{c}, \hat{x}).$$

Loss:

$$L = 2L_{\text{cls}} + 5L_{\text{reg}} + 2L_{\text{DIoU}}.$$

Inference:

$$x_T \rightarrow x_{T-s} \rightarrow \cdots \rightarrow x_0.$$

Post-processing:

NMS

removes redundant detections.

27 Integrated Conceptual Walkthrough

It is useful to follow one hypothetical object through the entire system.

Suppose a car is present in the LiDAR scene.

First, LiDAR measures many 3D points around the car:

$$(x_i, y_i, z_i).$$

The points are voxelized. The voxel encoder/backbone processes the occupied voxels and learns geometric features.

Those learned features are transformed into a BEV feature map. The car now corresponds to a spatial region in this top-down representation.

During training, the ground-truth car box is corrupted with Gaussian noise. The resulting proposal may be shifted, rotated, or have altered dimensions.

The proposal is checked against the LiDAR point cloud. If too few LiDAR points fall inside its physical BEV footprint, the proposal is rejected and resampled.

For an accepted proposal, its corresponding region is extracted from the BEV feature map. This gives the decoder local scene information.

The diffusion timestep is embedded and supplied to the decoder. The decoder therefore knows both what the local scene looks like and how noisy the current proposal is.

The decoder predicts class and geometric corrections. Matching determines which prediction corresponds to the ground-truth car, and the classification, regression, and DIOU losses train the network.

At inference, no ground-truth car box is available. Instead, the detector starts from random boxes. The same learned mechanism progressively denoises and refines them. Candidate boxes move toward regions supported by the LiDAR-derived BEV features.

Several candidates may converge to the same car, so NMS removes redundant predictions.

This is the full logic of the method.

28 Most Important Conceptual Comparison

A traditional anchor-based detector follows

predefined anchors $\rightarrow$ candidate regions $\rightarrow$ refinement $\rightarrow$ detections.

Diff3Det follows

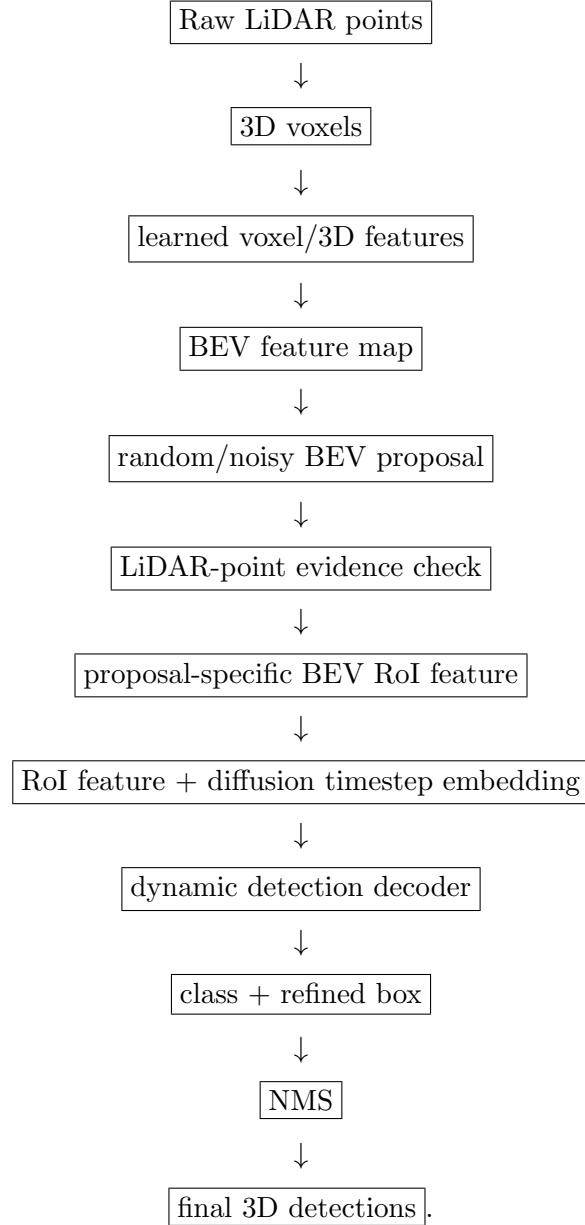
random/noisy boxes $\rightarrow$ diffusion denoising $\rightarrow$ candidate object boxes $\rightarrow$ detections.

The fundamental change is therefore:

manually specified proposal mechanism $\rightarrow$ learned generative proposal mechanism.

29 Final End-to-End Mental Model

The complete representation flow is



The particularly important point is that the BEV proposal and the original LiDAR point cloud refer to the same physical horizontal coordinate system. Therefore the proposal can both select an RoI from the BEV feature map and be checked for the number of LiDAR points in its footprint.

The method consequently combines efficient voxel/BEV feature extraction with diffusion-based learned proposal generation and a detection decoder that performs the final semantic and geometric prediction.